# Parallel Applications – Laboratory 5

Evaluating Threads, Pipes, Shared Memory, and Sockets on Image Sharpening

Skanda P (CS24BT045) N Suhas (CS24BT019)

## 1 Overview and Methodology

We parallelized the three-stage PPM image-sharpening pipeline from Laboratory 1—`S1` (3×3 neighborhood averaging/smoothing), `S2` (detail extraction, orig−smooth), and `S3` (sharpened clamping, clamp(orig+detail))—across single-machine multi-threading, multi-process IPC, and two-machine TCP sockets in C++17 (compiled with `g++ -O3 -Wall -Wextra -std=c++17 -pthread`). All numbers and plots in this report come directly from `src/results.csv`, generated by `src/run_experiments.sh` over all seven PPM images (`1.ppm`–`7.ppm`, from 450 × 180 to 3750 × 2470 pixels) using `ITERS=1000` iterations and `REPS=3` repetitions per configuration.

**Part I and Fair Single-Thread Baseline.** `part1/part1.cpp` preserves our Laboratory 1 single-pass implementation (`make part1` creates `output_part1.ppm`). Because a single pass is I/O-bound, Part II repeats `S1`, `S2`, and `S3` 1000 times after reading the input file once. For an accurate speed-up baseline, `baseline_bench.cpp` in `part1/` reads and allocates the image once and executes the exact same per-pixel kernels and pass counts (`1000` iterations) without threads, IPC, or checksums. Using `keep()` compiler barriers ensures intermediate passes not written to disk cannot be dead-code eliminated. We also fixed `common/libppm.cpp` so that after parsing the P6 `maxval` whitespace delimiter, the first raw pixel byte is read directly with `read_stream.get()` instead of skipping whitespace or blocking on `0x23` (`'#'`).

**Two Compute-Intensive Variants.** With `ITERS=1000`, file read + write time (Total − Compute) drops to only **0.25–0.39%** of total runtime across all seven images (5.2–5.4 ms on `1.ppm` and 462.7–473.8 ms on `7.ppm`), making execution overwhelmingly compute- and IPC-bound:

- **Variant 1:** All 1000 runs of `S1` stream pixel-by-pixel to `S2`, which streams to `S3`; the 1000th `S3` output is written to disk.
- **Variant 2:** `S1` runs 1000 times on the input image and streams every 100th result (10 frames) to `S2`; for each received frame, `S2` runs 100 passes and streams its 100th result to `S3`; for each received `S2` frame, `S3` runs 100 passes and writes the final frame to disk.

## 2 Implementation Details

### 2.1 Common Pipeline, Streaming, and Batching

All stage logic lives once in `common/stages.h` and `common/common_pipeline.h`, parameterized by a uniform channel interface (`put`, `flush`, `get`). No stage waits for an entire image before sending to the next stage, except Variant 2's `S2` and `S3`, which must buffer one received frame to run 100 local passes over it.

As a deliberate performance optimization, pixels are transmitted in small batches rather than flushing one 8-byte message per syscall or atomic update:

- **256 pixels** per publish/chunk on the lock-free atomic ring (Part II.1) and shared-memory ring (Part II.3) to avoid cache-line ping-ponging and amortize semaphore syscalls.
- **512 messages** (512 × 8 B = 4096 B = `PIPE_BUF`) on `FdWriter` for pipes (Part II.2) and TCP sockets (Part III), reducing `write()` system-call frequency while remaining strictly streamed mid-frame.

### 2.2 Part II Parallel Architectures and Makefile Conventions

- **Part II.1 (3 Threads + Atomics):** `run_part2_1` spawns three `std::thread` workers connected by two `AtomicChan` single-producer/single-consumer ring buffers ($2^{14}$ = 16,384 slots). Synchronization uses `std::atomic<size_t>` with `memory_order_acquire` and `memory_order_release`, producer/consumer-private cached indices, and `cpu_relax()`/`sched_yield()` backoff. Following the specification, the `S3`

thread (th3) calls write_ppm_file directly after stage3, recording the compute-end timestamp prior to writing so Compute Time stays separate from Write Time.

- **Part II.2 (3 Processes + Pipes):** run_part2_2 creates two anonymous POSIX pipes (pipe(), expanding kernel buffer capacity to 1 MiB via F_SETPIPE_SZ where supported) and fork()s children for S1 and S2 while the parent runs S3 and file output. Unused pipe ends are closed in each process, and write_fully/read_fully handle partial I/O and EINTR.
- **Part II.3 (3 Processes + Shared Memory + Semaphores):** Although the assignment's makefile bullet writes "multi-thread", the Part II.3 body text specifies *3 different processes that communicate via shared memory*; our implementation follows the body text. run_part2_3 maps a single anonymous shared region via mmap(..., MAP_SHARED | MAP_ANONYMOUS) holding two ShmQueue rings (64 slots of SHM_CHUNK = 256 pixels each), synchronized by process-shared POSIX counting semaphores (sem_init(..., pshared=1) for empty and full). Parent and children install PR_SET_PDEATHSIG (SIGKILL) and fatal parent notification so no process hangs on a broken pipe or semaphore.
- **Makefile Targets:** Following the assignment text literally, make part2_1_variantX creates output_part2_1_variantX.ppm and copies it to output_part2_2_variantX.ppm. The part2_3 target compiles and runs both variants (part2_3_variant1 and part2_3_variant2 also work individually), and part3_2_* mirrors part3_1_* as instructed.

# 3 Verification of Pixel Order and Integrity

To prove that every pixel is received unmodified and in exact raster order on every link (S1→S2, S2→S3, and A→B), each pixel is wrapped in an explicit 8-byte PixelMsg:

```
struct PixelMsg { int32_t seq; uint8_t r, g, b; uint8_t pad; };
```

with zeroed padding so raw struct transfers over pipes, shared memory, and sockets never leak uninitialized bytes.

1. **Order Proof:** Within each streamed image, Sender tags pixels sequentially with seq $\in \{0, \ldots, WH-1\}$. On every read, Receiver::get asserts m.seq == expected_seq and immediately aborts on any gap, duplication, or reordering.
2. **Data Integrity Proof:** Both Sender and Receiver incrementally fold every transmitted (seq, r, g, b) into a 32-bit FNV-1a checksum (FNV_INIT = 2166136261u, prime 16777619u). At end-of-stream, Sender::finish() emits a trailer message carrying the sender's checksum in seq and sentinel RGB bytes (0xC5, 0x5A, 0x3C); Receiver::finish() verifies the sentinel and aborts on any checksum mismatch.
3. **End-to-End Equivalence and Fault Injection:** Across all 56 rows in results.csv, every parallel output hash matches the single-thread baseline bit-for-bit (output_ok = yes; e.g., 709e28fa on 1.ppm). In addition, make fault_test compiles Part II.1, II.2, II.3, and Part III.1 with -DINJECT_FAULT, flipping 1 bit of bad.r on the 12,345th transmitted pixel after hashing; all four implementations detect the corruption and abort as expected.

# 4 Experimental Results: Parts I and II

Tables 1 and 2 report the mean Compute Time (ms, over 3 repetitions at ITERS=1000) and speed-up relative to the same-variant single-thread baseline across all seven images. Figures 1 and 2 visualize the speed-ups and scaling behavior.

**Observations on Runtime and Speed-Up.**

- **Why Variant 1 slows down (0.76–0.89×):** Passing every pixel of all 1000 passes over two IPC links ships $1000 \times W \times H \times 8$ B per hop—reaching 74.1 GB per link (148.2 GB total) on 7.ppm. Serializing PixelMsg, computing FNV-1a twice per hop, and synchronizing across cores costs more than the lightweight pixel arithmetic, whereas the single-thread Variant 1 baseline fuses S1→S2→S3 in CPU registers with zero IPC. Threads + atomics (0.83–0.89×) consistently beat pipes (0.76–0.81×) and semaphores (0.76–0.80×) in Variant 1 because user-space ring updates avoid kernel syscalls.

| Image | Resolution | Pixels | Baseline (ms) | Part II.1 (ms / Spd) | Part II.2 (ms / Spd) | Part II.3 (ms / Spd) |
|---|---|---|---|---|---|---|
| 1.ppm | 450 × 180 | 81,000 | 1,429.1 | 1,623.9 (0.88×) | 1,791.7 (0.80×) | 1,818.6 (0.79×) |
| 2.ppm | 786 × 393 | 308,898 | 5,515.3 | 6,210.9 (0.89×) | 6,846.5 (0.81×) | 6,928.3 (0.80×) |
| 3.ppm | 1024 × 575 | 588,800 | 10,246.5 | 12,064.7 (0.85×) | 13,085.2 (0.78×) | 13,217.7 (0.78×) |
| 4.ppm | 1280 × 910 | 1,164,800 | 19,963.0 | 23,742.0 (0.84×) | 25,954.4 (0.77×) | 26,047.6 (0.77×) |
| 5.ppm | 1500 × 1000 | 1,500,000 | 25,364.4 | 30,174.8 (0.84×) | 32,720.1 (0.78×) | 33,236.3 (0.76×) |
| 6.ppm | 2271 × 1500 | 3,406,500 | 57,203.0 | 67,972.2 (0.84×) | 73,906.1 (0.77×) | 75,164.8 (0.76×) |
| 7.ppm | 3750 × 2470 | 9,262,500 | 154,904.1 | 185,820.2 (0.83×) | 202,943.4 (0.76×) | 204,092.4 (0.76×) |

Table 1: Variant 1 mean compute times and speed-ups (`ITERS=1000`, `REPS=3`). Ideal 3-stage limit = 3.0×.

| Image | Resolution | Pixels | Baseline (ms) | Part II.1 (ms / Spd) | Part II.2 (ms / Spd) | Part II.3 (ms / Spd) |
|---|---|---|---|---|---|---|
| 1.ppm | 450 × 180 | 81,000 | 1,661.5 | 1,426.9 (1.16×) | 1,043.7 (1.59×) | 1,344.2 (1.24×) |
| 2.ppm | 786 × 393 | 308,898 | 7,003.3 | 6,016.0 (1.16×) | 6,077.0 (1.15×) | 6,060.2 (1.16×) |
| 3.ppm | 1024 × 575 | 588,800 | 13,884.2 | 8,277.4 (1.68×) | 7,620.6 (1.82×) | 7,590.5 (1.83×) |
| 4.ppm | 1280 × 910 | 1,164,800 | 26,320.4 | 16,151.9 (1.63×) | 14,903.9 (1.77×) | 14,842.7 (1.77×) |
| 5.ppm | 1500 × 1000 | 1,500,000 | 33,926.5 | 27,540.8 (1.23×) | 29,777.3 (1.14×) | 27,541.4 (1.23×) |
| 6.ppm | 2271 × 1500 | 3,406,500 | 76,675.8 | 65,727.1 (1.17×) | 64,761.0 (1.18×) | 63,523.0 (1.21×) |
| 7.ppm | 3750 × 2470 | 9,262,500 | 208,085.2 | 180,255.2 (1.15×) | 173,510.8 (1.20×) | 172,610.6 (1.21×) |

Table 2: Variant 2 mean compute times and speed-ups (`ITERS=1000`, `REPS=3`). All outputs verified identical to baseline.

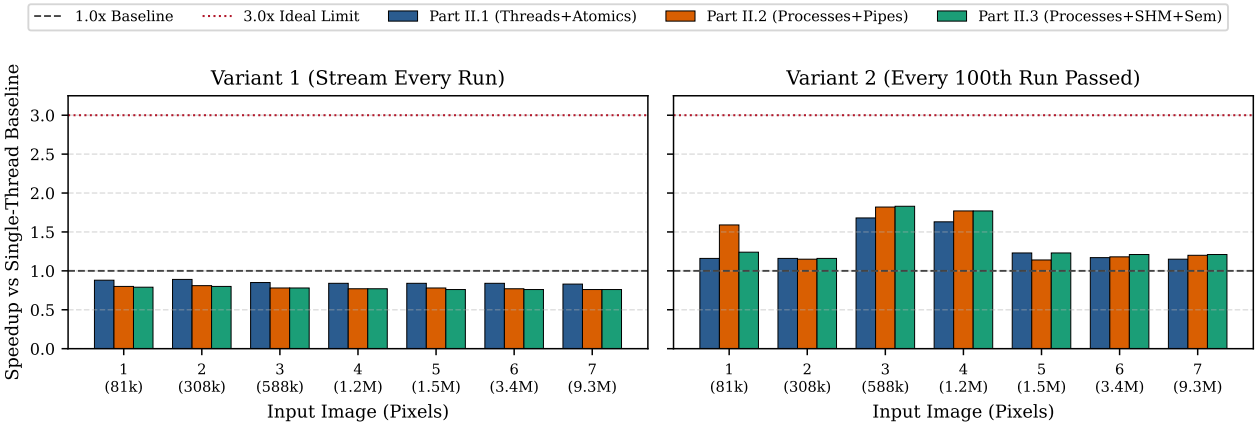



Figure 1: Speed-up across all seven images for Variant 1 (left) and Variant 2 (right), showing the 1.0× baseline and 3.0× ideal 3-stage pipeline limit.

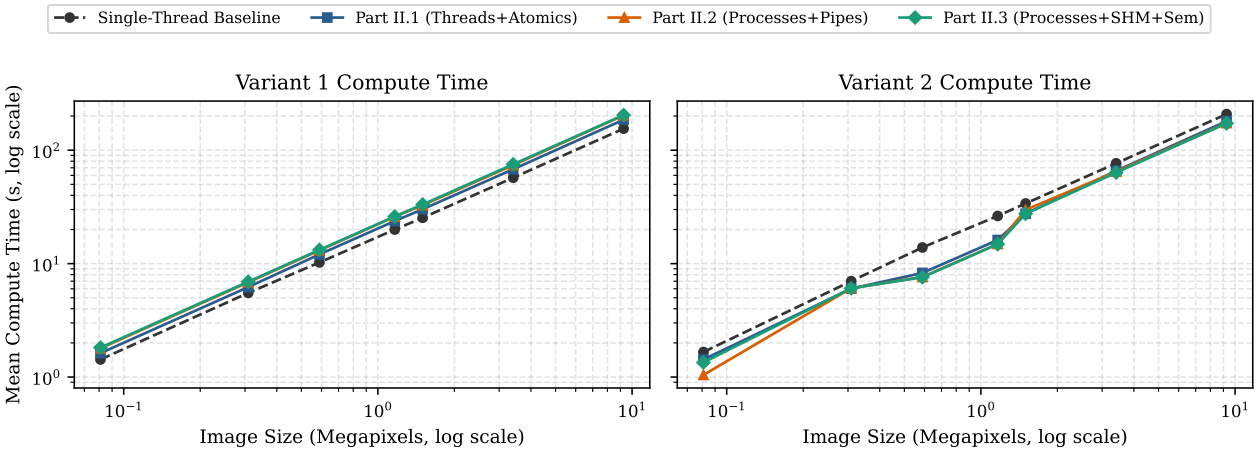



Figure 2: Log–log scaling of mean compute time with image pixel count for Variant 1 (left) and Variant 2 (right).

- **Why Variant 2 achieves speed-up (1.14–1.83×):** Variant 2 reduces IPC volume by 100× (only 10

frames cross each link), allowing all three cores to execute their 1000 passes concurrently. Also note the single-thread Variant 2 baseline is 16–35% slower than Variant 1 because it writes and reads full-frame `PixelMsg` arrays between 100-pass blocks rather than fusing stages in registers.

- **Pipeline stage imbalance and memory-bandwidth saturation:** Even without IPC overhead, speed-up is bounded well below 3.0× because `S1` performs a 3×3 neighborhood average (9 reads + integer division per channel) while `S2` and `S3` perform one subtraction or addition+clamp per channel. In our Laboratory 1 single-pass measurements, `S1` took ≈ 60–64% of compute time (13.63 ms vs 4.39 + 4.90 ms on `1.ppm`; 1087.0 ms vs 294.0 + 313.8 ms on `7.ppm`). Thus the slowest stage (`S1`) caps pipeline speed-up near 1/0.60 ≈ 1.67–1.8×, matching the observed peak of **1.68–1.83**× on `3.ppm` and `4.ppm`. On larger images (`5.ppm`–`7.ppm`), full-frame `PixelMsg` buffers (12–74 MB each) exceed L3 cache, so three concurrent workers contend on DRAM bandwidth and settle at **1.14–1.23**×.
- **I/O vs. compute scaling:** Both compute and I/O scale linearly with pixel count (81k to 9.26M pixels, a 114× increase producing a ≈ 108–128× compute increase). Without the 1000-iteration loop, disk read/write (≈ 8 ms on `1.ppm`, ≈ 470 ms on `7.ppm`) would dominate a single pass (≈ 2 ms to ≈ 160 ms compute), masking parallel speed-ups.

# 5 Part III: Distributed Execution Across Two Computers

## 5.1 Protocol and Usage Convention

Part III splits the pipeline over TCP sockets (`run_part3_A` as server, `run_part3_B` as client, with `TCP_NODELAY` enabled). Since the specification does not specify how Computer B discovers Computer A's address, our convention passes A's LAN address (obtained via `ip a`) and port via `make` variables (`PORT=12345` default, opened with `sudo ufw allow 12345/tcp`):

```
Computer A: make part3_1_variantX_A
Computer B: make part3_1_variantX_B IP=<A's IP>
```

(and identically for `part3_2_variantX_A/B`). Once connected (with a 60-second connect retry limit on B), A sends a 16-byte network-order header $\{W, H, \text{iters}, \text{fnv(img)}\}$ and the raw original image (which B needs for `S2`/`S3`), followed by the `PixelMsg` stream with `seq` converted via `htonl`/`ntohl`. After `Receiver::finish()` passes, B sends a `'K'` confirmation byte before closing so A exits non-zero if B aborts.

## 5.2 Comparing the Two Partitions

- **Part 3.1 (`A = S1+S2, B = S3`):** Computer A executes both the heavy 3×3 smoothing (`S1`, ∼ 60–64%) and detail extraction (`S2`, ∼ 18%), carrying roughly ∼ 80% of compute per pass while Computer B performs only clamping addition (`S3`, ∼ 20%) and waits idle most of the time.
- **Part 3.2 (`A = S1, B = S2+S3`):** Computer A performs `S1` (∼ 60–64%) and Computer B performs `S2+S3` (∼ 36–40%).
- **Which is better and why: Part 3.2 (`A = S1, B = S2+S3`) is the better partition** when both hosts have comparable CPUs, because moving `S2` to B balances CPU load (∼ 60/40 instead of ∼ 80/20), reducing the bottleneck stage time by ∼ 20%. Network payload per pixel is identical in both splits (one 8-byte `PixelMsg` stream per transferred frame, 100× smaller in Variant 2 than Variant 1). *Note:* `src/results.csv` records only the single-machine Parts I and II benchmark suite (`RUN_PART3=0`); two-computer LAN measurements were not recorded in `results.csv`.

# 6 Ease of Implementation and Debugging

- **Threads + Atomics (II.1):** Hardest to design and debug subtly. Correct lock-free SPSC rings require careful acquire/release pairing, alignment (`alignas(64)`) to prevent false sharing, and cached indices; bugs manifest as rare, non-deterministic data races.
- **Processes + Pipes (II.2):** Easiest overall. Kernel-managed blocking handles synchronization automatically once unused pipe ends are closed and partial reads/writes (`read_fully`) are looped properly.

| Mechanism | Sync Primitive | Kernel Crossing | Implementation | Debugging Difficulty |
|---|---|---|---|---|
| II.1 Threads | `std::atomic` (acq/rel) | None (user space) | Moderate–Hard | Hardest (races, memory order) |
| II.2 Pipes | Blocking `read`/`write` | Yes (per batch) | Easiest | Easy (watch unclosed FDs) |
| II.3 SHM + Sem | POSIX `sem_wait`/`post` | Per 256-px chunk | Moderate | Moderate–Hard (sem deadlocks) |
| III Sockets | TCP stream + `htonl` | Yes (network stack) | Moderate | Hardest (firewall, partial I/O) |

Table 3: Comparison of synchronization, kernel overhead, and development/debugging trade-offs.

- **Processes + SHM + Semaphores (II.3):** Straightforward ring indexing, but unmatched `sem_wait` or `sem_post` calls or a crashing child can deadlock the pipeline; using `PR_SET_PDEATHSIG` on children and `SIGKILL` on `fatal()` prevents hangs.
- **TCP Sockets (III):** Requires explicit framing, endianness conversion, partial `read()` buffering, two-machine coordination (`ip a`, `ufw`), and explicit peer abort propagation (`'K'` byte).

# 7 Conclusions

- Repeating the transform 1000 times reduces file I/O to 0.25–0.39% of runtime across all seven images, isolating parallel compute and IPC costs.
- Streaming every pass (Variant 1) achieves 0.76–0.89× (slower than baseline) because shipping up to 74.1 GB per link outweighs lightweight pixel arithmetic. Passing every 100th pass (Variant 2) cuts IPC by 100× and reaches up to **1.83**× speed-up, capped by `S1`'s $\sim$ 60–64% compute share and DRAM bandwidth on multi-megapixel images.
- All channels preserve exact raster order and bit-identical output (`output_ok = yes` on all 56 runs in `results.csv`; single-bit errors caught by `make fault_test`).
- For two computers (Part III), splitting as **Part 3.2 (`A=S1, B=S2+S3`)** balances CPU compute better than Part 3.1 with identical network traffic.

**Reproducing.** From `lab5_new/src/`, `make all` builds and runs Parts I and II, `make fault_test` verifies corruption detection, and `./run_experiments.sh` regenerates `results.csv`.